



## CAPÍTULO 7

# Alocação dinâmica de memória

Como um programa pede e devolve memória em tempo de execução — e por que vetores e matrizes deixam de ter tamanho fixo.



# O que vamos ver

## 01 Uso da memória

heap e pilha crescem em direções opostas; malloc devolve um endereço

---

## 02 Alocação de vetores

malloc( $n * \text{sizeof}(\text{tipo})$ ) do tamanho certo, decidido em tempo de execução

---

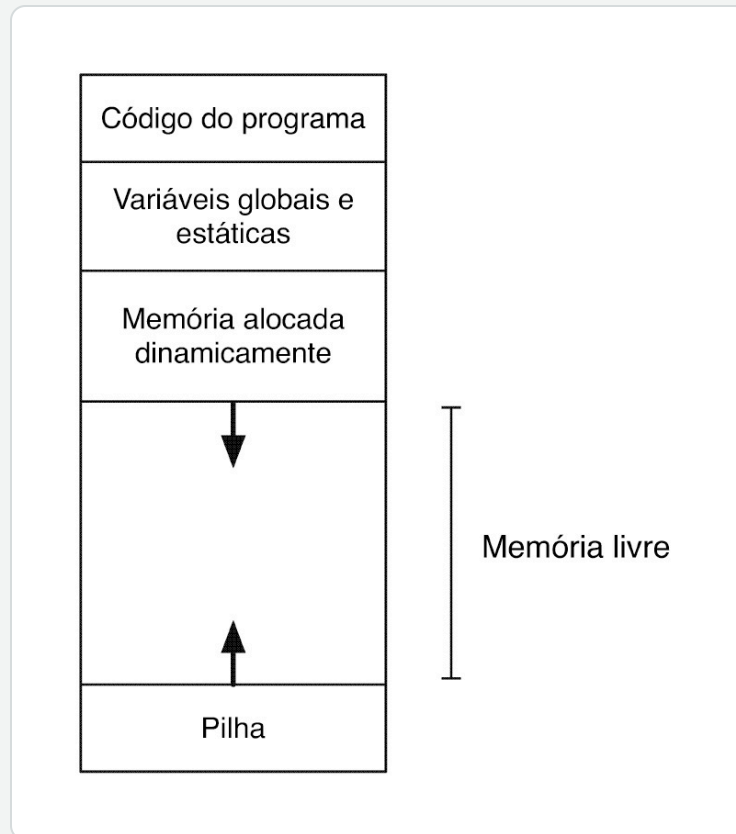
## 03 Alocação de matrizes

vetor único (row-major) ou vetor de ponteiros — cada um com seu jeito de liberar

---

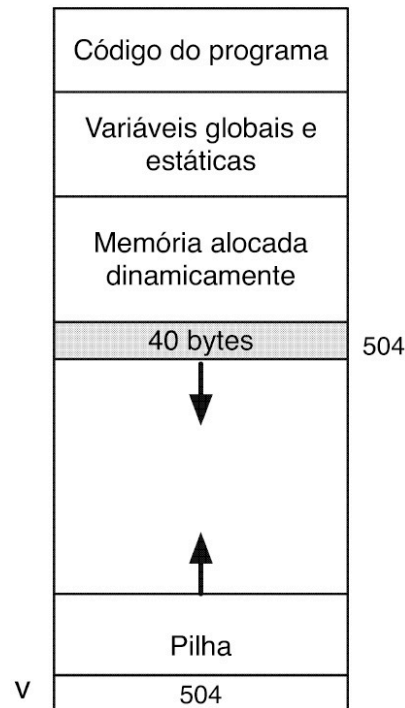
# Onde a memória de um programa vive

Heap e pilha crescem em direções opostas, dividindo entre si a memória livre.



# malloc devolve um endereço

Um ponteiro comum, na pilha, guarda o início do bloco alocado no heap.



# Quem não devolve, vaza

Memória de malloc não desaparece sozinha — só um free explícito a devolve.

## **free(v)**

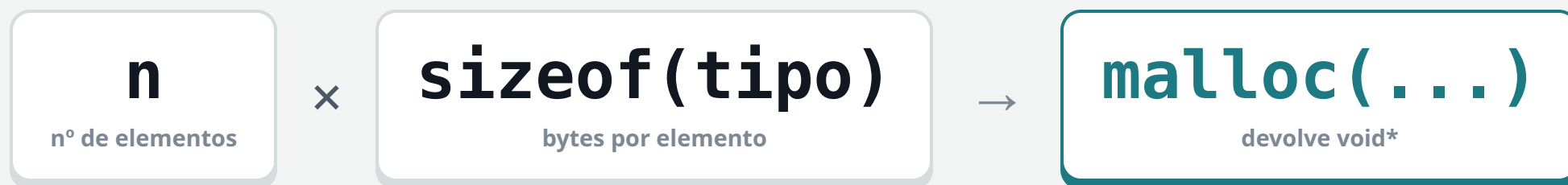
Devolve o bloco ao heap. Usar v depois disso é *use-after-free*.

## **– sem free –**

O bloco fica preso, inacessível, até o programa terminar: um vazamento de memória.

# O tamanho vira uma conta, não uma constante

`n` pode ser lido em tempo de execução — um vetor comum não permite isso.



# Nem todo vetor sobrevive ao retorno

Devolver o endereço de um vetor da pilha é um ponteiro pendurado — do heap, não.

```
int v[10]
```

pilha – morre  
ao retornar,  
ponteiro  
pendurado

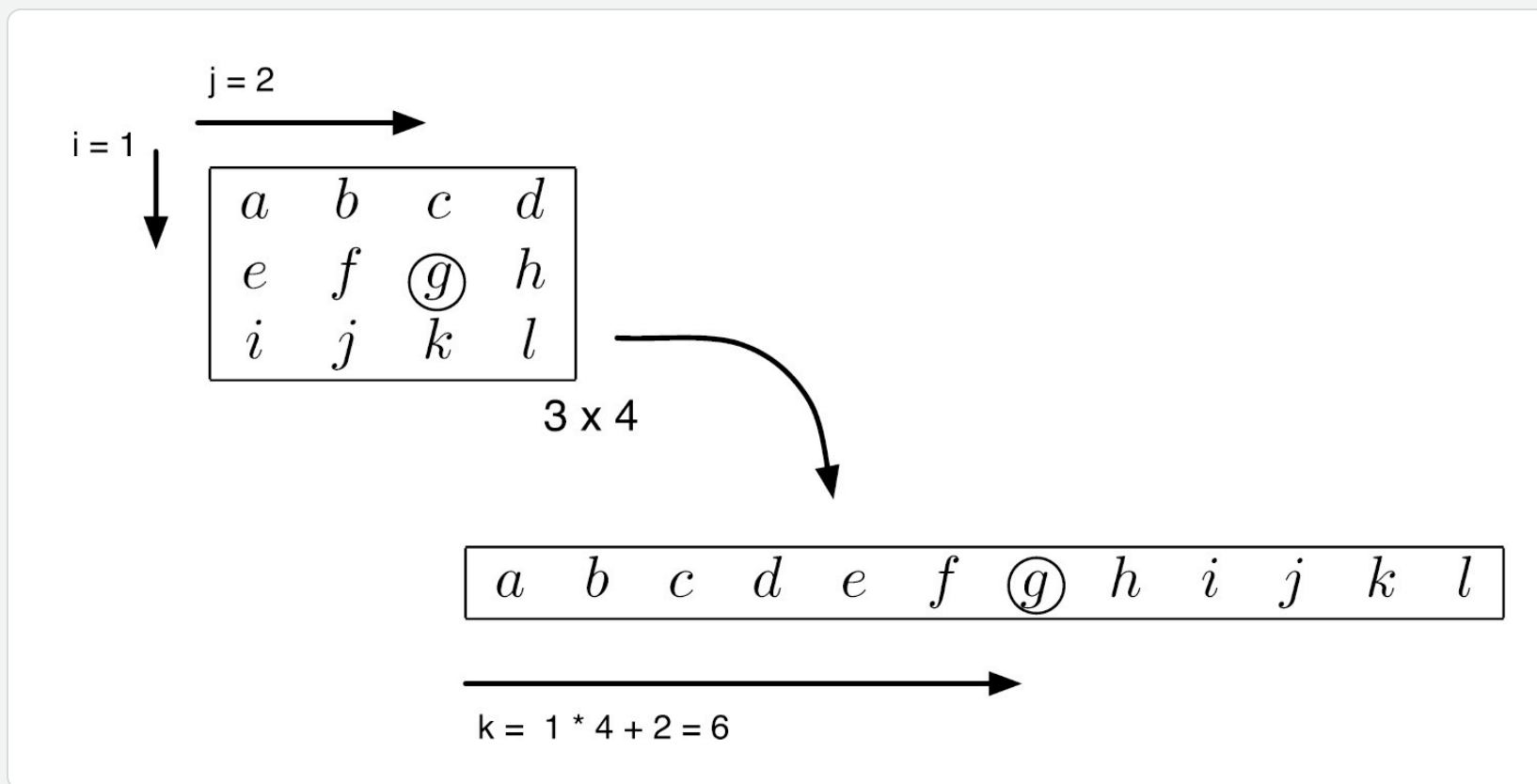
VS

```
malloc(...)
```

heap –  
sobrevive ao  
retorno, uso  
seguro

# Matriz como um único vetor

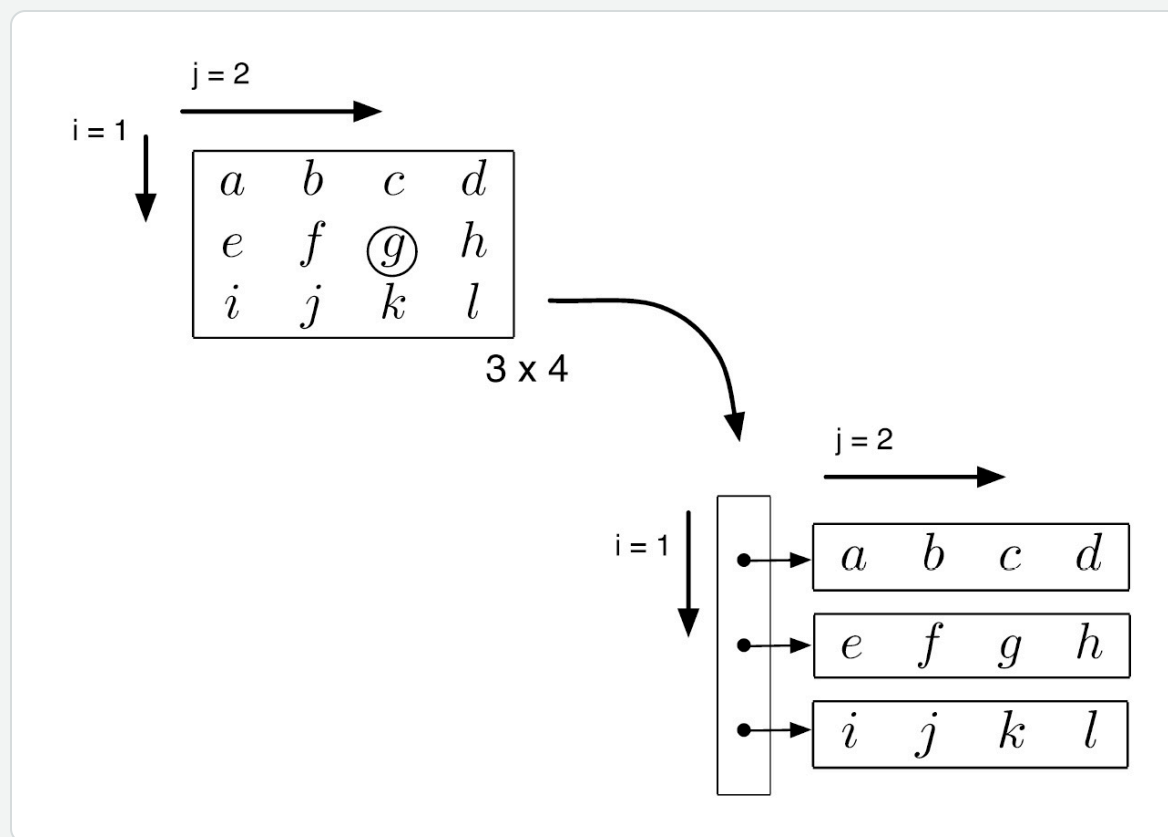
Row-major: o elemento da linha  $i$ , coluna  $j$ , mora na posição  $i * N + j$ .





# Matriz como vetor de ponteiros

Cada linha é seu próprio bloco — `matrix[i][j]`  
direto, sem contas de índice.



# Um bloco só, ou vários — e liberar cada um

## Vetor único

Um malloc, um free para a matriz inteira. Acesso via  $i * N + j$ .

## Vetor de ponteiros

$N + 1$  blocos independentes — libera-se cada linha, na ordem inversa da alocação.

AGORA É COM VOCÊ

# Explore os exemplos, no seu ritmo

Código-fonte, explicação linha a linha e simulação da execução passo a passo — sem precisar compilar nada.

 **Notas de Aula e Exemplos** [thiagopx.github.io/ed/05\\_dynamic\\_allocation](https://thiagopx.github.io/ed/05_dynamic_allocation)

Depois, teste o que aprendeu no quiz do módulo.